I302 - Aprendizaje Automático y Aprendizaje Profundo

Trabajo Práctico 3: Redes Neuronales

Ana Paula Tissera

12 de mayo de 2025

Ingeniería en Inteligencia Artificial

Resumen

En este trabajo se abordó la clasificación de caracteres japoneses escritos a mano a partir de un dataset similar a MNIST. Se implementó desde cero una red neuronal feedforward con ReLU y softmax en la capa de salida, y se exploraron mejoras en el entrenamiento —SGD por minibatches, Adam, programación de la tasa de aprendizaje, regularización L_2, dropout, batch normalization y early stopping— así como distintas arquitecturas mediante búsquedas *greedy* por limitaciones computacionales. La mejor configuración (modelo $M3$: capas de 768 y 384 neuronas) obtuvo un 70,6 % de *accuracy* y 2,08 de pérdida, superando tanto a la implementación base (60,2 %) como a la implementación propia y a la réplica en PyTorch con los mismos hiperparámetros (67,7 % y 69,0 % respectivamente).

Finalmente, se generaron y guardaron en un `csv` las probabilidades de las 49 clases para un conjunto de evaluación externa.

1. Introducción

En este trabajo abordamos la clasificación de caracteres japoneses a partir de un conjunto de datos similar a MNIST, compuesto por imágenes en escala de grises de 28×28 píxeles y 49 clases posibles. La entrada del algoritmo consiste en la matriz de píxeles normalizada al rango $[0, 1]$ de cada imagen, y la salida es un vector de probabilidades a posteriori para cada una de las 49 categorías.

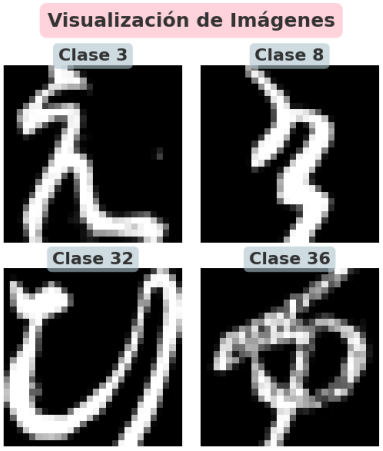


Figura 1: Visualización de ejemplo

Para abordar este problema, se implementó desde cero una red neuronal feedforward, en primera instancia con dos capas ocultas (100 y 80 neuronas, respectivamente), utilizando la función de activación ReLU en las capas intermedias y softmax en la capa de salida, y empleando la entropía cruzada como función de costo. Después, se exploraron diversas mejoras en el proceso de entrenamiento —incluyendo descenso de gradiente estocástico por mini-batches, optimizador Adam, programación de tasa de aprendizaje (lineal y exponencial), regularización L2, early stopping, dropout y batch normalization, así como diversas arquitecturas— para evaluar su impacto en la convergencia y la capacidad de generalización. Finalmente, se replicaron las arquitecturas y los mejores hiperparámetros en PyTorch para comparar la performance de los modelos implementados manualmente con los entrenados en un framework estándar, y se generaron predicciones de probabilidades sobre un conjunto de datos de evaluación externa.

2. Métodos

En este trabajo se implementaron desde cero e investigaron los siguientes algoritmos y técnicas de entrenamiento para la clasificación de caracteres escritos a mano:

2.1. Red neuronal feedforward

Sea $\mathbf{x} \in R^{784}$ el vector de píxeles de una imagen 28×28. En una primera instancia, definimos una red de $L = 3$ capas intermedias (dos ocultas y una de salida) con parámetros $\{W^{(l)}, b^{(l)}\}_{l=1}^{L}$. Para más información sobre la actualización de los parámetros revisar la sección siguiente (2.2).

Las funciones de activación de las capas ocultas son $f^{(1)}, f^{(2)} =$

$$\mathrm{ReLU}(z) = \mathrm{máx}(0, z) \tag{1}$$

y para la capa de salida, $f^{(3)}$ es la función softmax:

$$\hat{y}_k = \frac{\exp(z_k^{(3)})}{\sum_{j=1}^{49} \exp(z_j^{(3)})}. \tag{2}$$

Como función de costo se usa la entropía cruzada en un batch de tamaño m:

$$\mathcal{L}(\theta) \;=\; -\frac{1}{m} \sum_{i=1}^{m} \sum_{k=1}^{49} y_{i,k} \, \log \hat{y}_{i,k}, \tag{3}$$

donde $y_{i,k}$ es la etiqueta one-hot del ejemplo i.

2.2. Backpropagation

Para ajustar los parámetros $\{W^{(l)}, b^{(l)}\}$ se utiliza el algoritmo de *backpropagation*, que aplica la regla de la cadena para computar los gradientes de la función de costo $\mathcal{L}$ (3) respecto a cada $W^{(l)}$ y $b^{(l)}$. El procedimiento se divide en dos fases:

Forward pass Dada la entrada $\mathbf{x}$, recorremos la red capa a capa para obtener las activaciones:

$$\mathbf{a}^{(0)} = \mathbf{x},$$
$$\mathbf{z}^{(l)} = W^{(l)} \, \mathbf{a}^{(l-1)} + b^{(l)},$$
$$\mathbf{a}^{(l)} = \begin{cases} \mathrm{ReLU}(\mathbf{z}^{(l)}) \;(1) & l = 1, \ldots, L-1, \\ \mathrm{softmax}(\mathbf{z}^{(L)})_k \;(2) & l = L. \end{cases}$$

Donde la salida final $\hat{\mathbf{y}} = \mathbf{a}^{(L)}$ es un vector de probabilidades.

Backward pass Ahora se propaga el error desde la salida hasta la entrada:

1. *Error en la capa de salida.* Con la combinación softmax+cross-entropy se simplifica el gradiente a

$$\delta^{(L)} = \mathbf{a}^{(L)} - \mathbf{y},$$

donde $\mathbf{y}$ es la etiqueta one-hot.

2. *Error en capas intermedias.* Para $l = L-1, \ldots, 1$,

$$\delta^{(l)} = \left(W^{(l+1)}\right)^T \delta^{(l+1)} \;\circ\; f'(\mathbf{z}^{(l)}),$$

con $f'(z) = 1$ si $z > 0$ y 0 en caso contrario (derivada de ReLU).

3. *Cálculo de gradientes.* Una vez obtenidos los $\delta^{(l)}$, computamos

$$\nabla_{W^{(l)}} \mathcal{L} = \delta^{(l)} \left(\mathbf{a}^{(l-1)}\right)^T, \quad \nabla_{b^{(l)}} \mathcal{L} = \delta^{(l)}.$$

Estos gradientes se utilizan luego en los pasos de actualización con el método de optimización elegido (SGD, Adam, etc.), completando una iteración de entrenamiento.

2.3. Descenso de gradiente estocástico (SGD) por minibatches y Adam

En lugar de computar el gradiente sobre todo el conjunto de datos, el *SGD* por minibatches actualiza

$$\theta \leftarrow \theta - \eta \, \nabla_\theta \mathcal{L}_{\mathcal{B}},$$

donde $\mathcal{B}$ es un subconjunto aleatorio en cada iteración, lo que reduce el costo computacional y agrega ruido útil para escapar de óptimos locales.

El optimizador Adam mejora SGD acumulando momentos de primer y segundo orden:

$$m_t = \beta_1 \, m_{t-1} + (1 - \beta_1) \, g_t,$$
$$v_t = \beta_2 \, v_{t-1} + (1 - \beta_2) \, g_t^2,$$
$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t},$$
$$\theta =\leftarrow \theta - \eta \, \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}.$$

con $g_t = \nabla_\theta \mathcal{L}_{\mathcal{B}}$.

2.4. Programación de la tasa de aprendizaje

Para mejorar la convergencia, variamos la tasa η_t a lo largo del entrenamiento. Se exploraron dos esquemas de decaimiento de η_t:

- *Lineal:* $\eta_t = \eta_0\,(1 - t/T)$.
- *Exponencial:* $\eta_t = \eta_0\,\exp(-k\,t)$.

Estos esquemas permiten empezar con pasos grandes para corregir el sesgo inicial y después refinar con pasos más chicos, reduciendo el riesgo de oscilar alrededor del mínimo.

2.5. Regularización y técnicas de mejora

Para controlar el sobreajuste y favorecer la generalización se implementaron:

- *Regularización L_2:* $\mathcal{L}_{\text{reg}} = \mathcal{L} + \frac{\lambda}{2}\|\theta\|^2$. Esto penaliza pesos grandes y suaviza la solución.
- *Dropout:* Durante el entrenamiento se desactivan aleatoriamente neuronas de cada capa oculta con probabilidad p, aplicando un vector máscara $\mathbf{d} \sim \text{Bernoulli}(p)$ a las activaciones:

  $$\tilde{\mathbf{a}}^{(l)} = \mathbf{d} \circ \mathbf{a}^{(l)}.$$

  Este procedimiento equivale a entrenar una combinación de subredes, lo que limita las co-adaptaciones excesivas.
- *Batch normalization:* se normaliza $\mathbf{z}^{(l)}$ por mini-batch antes de la activación para mantener medias cero y varianzas unitarias, acelerando la convergencia.
- *Early stopping:* Se detiene el entrenamiento cuando la pérdida de validación no mejora tras n épocas, evitando iteraciones inútiles que incrementan la varianza y la probabilidad de sobreajustar.

2.6. Implementación en PyTorch

Se replicó la misma arquitectura y los mejores hiperparámetros hallados en la implementación manual dentro de un modelo en PyTorch, con el fin de comparar la eficiencia computacional y la calidad de la convergencia entre ambos entornos.

2.7. Conjunto de datos

El conjunto completo de imágenes se dividió inicialmente en un 80 % para desarrollo y un 20 % para prueba, aplicando una mezcla aleatoria previo a la partición. A su vez, el subconjunto de desarrollo se subdividió en un 80 % para entrenamiento y un 20 % para validación, de modo que las proporciones finales sobre el total original fueron:

Entrenamiento: $0{,}8 \times 0{,}8 = 0{,}64$ (64 %),
Validación: $0{,}8 \times 0{,}2 = 0{,}16$ (16 %),
Prueba: 0,20 (20 %).

Antes de ingresar las imágenes a la red, cada píxel, originalmente en el rango $[0, 255]$, se normalizó a $[0, 1]$ mediante la transformación

$$x_{\text{norm}} = \frac{x}{255}. \tag{4}$$

También se analizó la distribución de las clases y, dado que la proporción de muestras por clase era aproximadamente uniforme, no se aplicó ninguna técnica de rebalanceo ni de aumento de datos.

2.8. Selección de hiperparámetros

Para elegir la configuración óptima se implementó un *greedy search* secuencial (ver `utils.py` para la implementación) sobre el conjunto de parámetros y arquitecturas:

1. Se definió un espacio de arquitecturas, es decir, una lista de tamaños de capa con distintas profundidades y complejidades, optimizadores {`'gd'`, `'mb-sgd'`,

`'adam'`}, tasa de aprendizaje $\eta \in \{0{,}001,\ 0{,}005,\ 0{,}01,\ 0{,}05,\ 0{,}1,\ 0{,}15,\ 0{,}2\}$, tamaño de mini-batch $m \in \{1,\ 16,\ 32,\ 64,\ 128,\ 256\}$ (o `None` para entrenamiento full-batch), λ de regularización $L_2 \in \{0,\ 10^{-3},\ 10^{-2},\ 10^{-1}\}$, probabilidad de *dropout* $p \in \{0,\ 0{,}15,\ 0{,}2,\ 0{,}25,\ 0{,}3\}$, uso o no de *batchnorm*, activación de *early stopping*, *patience* $\in \{5,\ 10,\ 15,\ 20,\ 30,\ 50\}$, y programación de learning rate {`'none'`, `'linear'`, `'exp'`}, con tasa de decaimiento (*decay_rate*) $\in \{0{,}01,\ 0{,}05,\ 0{,}1,\ 0{,}25\}$ y tasa de learning rate mínima (*lr_min*) $\in \{0{,}001,\ 0{,}005,\ 0{,}01,\ 0{,}02\}$.

2. A partir de un orden de parámetros predeterminado, se fijaron en cada paso todos los valores óptimos hallados previamente y se exploró únicamente el parámetro actual. Dado que la naturaleza *greedy* del método no garantiza la optimalidad global, en ciertos puntos del proceso se re-ejecutaron entrenamientos con algunas arquitecturas o valores previamente seleccionados, con el objetivo de confirmar la consistencia de los resultados y corregir posibles elecciones subóptimas antes de avanzar al siguiente parámetro. Esta implementación se puede ver en la sección 3.2 del `notebook`.

3. Cada configuración se entrenó sobre el conjunto de entrenamiento y se evaluó en el de validación, usando hasta 250 épocas y detención temprana si la pérdida de validación no mejoraba tras la `patience` definida.

2.9. Métricas de evaluación

Para comparar el rendimiento entre distintas configuraciones de red e hiperparámetros se utilizó como métrica principal la *accuracy*, definida por:

$$\text{Accuracy} = \frac{1}{n}\sum_{i=1}^{n} 1\big[\hat{y}_i = y_i\big], \tag{5}$$

donde $\hat{y}_i$ es la clase predicha para el ejemplo i, y_i es la clase verdadera, y n es la cantidad total de ejemplos en el conjunto evaluado.

Durante el entrenamiento se monitorearon también la pérdida (*loss*) de entrenamiento y validación, lo cual permitió aplicar criterios de *early stopping* para evitar sobreajuste.

Además, se analizó en detalle la distribución de errores mediante la matriz de confusión, definida como

$$C = \big[C_{ij}\big]_{i,j=1}^{49},$$

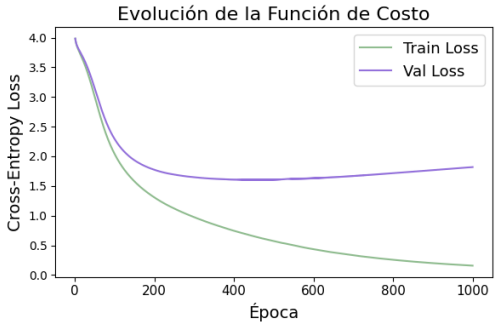
$$C_{ij} = \#\{\text{ muestras de la clase } i \text{ clasificadas como } j\}$$

3. Resultados

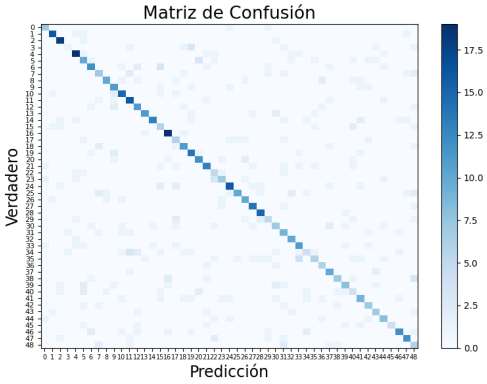
Comenzamos analizando el desempeño de la arquitectura inicial $M0$, compuesta por dos capas ocultas de 100 y 80 neuronas respectivamente, utilizando funciones de activación ReLU y una capa de salida softmax, tal como fue detallado en la Sección 2.1. Esta red fue entrenada con entropía cruzada como función de costo y sin aplicar todavía optimizadores ni técnicas de regularización o programación de tasa de aprendizaje.

En la Figura 2a se observa la evolución de la pérdida (*loss*) durante el entrenamiento. Inicialmente, ambas curvas (para los conjuntos de entrenamiento y validación) descienden de forma pronunciada hasta la época 200, lo cual es coherente con la fase de corrección de sesgo ("bias") donde el modelo ajusta rápidamente los patrones globales de los datos. Entre las épocas 200 y 600, la pérdida de validación se estabiliza alrededor de 1.5 mientras la de entrenamiento continúa bajando hacia 0.2, reflejando el punto de equilibrio bias–varianza en

el que la red aprende detalles específicos del entrenamiento sin mejorar la generalización. A partir de la época 600, se aprecia un ligero repunte en la curva de validación, signo de sobreajuste ("overfitting") al conjunto de entrenamiento.



(a) Evolución de la pérdida (*loss*) de entrenamiento y validación



(b) Matriz de confusión

Figura 2: Evaluación de $M0$ en el conjunto de validación

Al evaluar el modelo sobre el conjunto de validación, se obtuvieron los siguientes resultados:

$$\text{Accuracy}_{\text{val}} = 0{,}6088, \quad \text{Cross-Entropy Loss}_{\text{val}} = 1{,}8189.$$

Estos valores marcan el punto de partida, y sirven como referencia para la posterior optimización de hiperparámetros y arquitecturas mediante el procedimiento descrito en la Sección 2.8.

La Figura 2b muestra la matriz de confusión resultante para el conjunto de validación. Como se explicó en la Sección 2.9, esta herramienta permite visualizar con mayor detalle los errores cometidos por el modelo. Se aprecia que la mayoría de las predicciones se concentran en la diagonal principal, lo que indica un reconocimiento exitoso en la mayoría de las clases. No obstante, se ven errores repartidos fuera de la diagonal, especialmente entre clases morfológicamente parecidas, lo que indica que el modelo tiene dificultades para distinguir detalles sutiles.

Esto sugiere oportunidades de mejora para futuras versiones del modelo a través de técnicas de regularización, normalización o arquitecturas más profundas, que se exploraron a continuación.

3.1. Efecto de técnicas de optimización y regularización aisladas

Se implementaron las técnicas de optimización y regularización descritas en las secciones 2.3 2.4 y 2.5, en una primera instancia aplicándolas de forma individual sobre la arquitectura base $M0$. Para esto, se fijaron valores arbitrarios razonables de hiperparámetros (como $\eta = 0{,}01$, $\lambda = 10^{-1}$, $\text{dropout}_p = 0{,}2$, patience $= 5$) y se entrenó cada variante sin combinarla con otras, con el fin de tener una idea de su efecto sobre el aprendizaje en este problema.

La Figura 3 muestra la evolución de la función de costo (cross-entropy) durante el entrenamiento para cada método sobre la misma arquitectura $M0$. En el Cuadro 1 se presentan

las métricas finales de precisión (*Accuracy*) y entropía cruzada (CE) tanto en entrenamiento como en validación, permitiendo comparar el impacto de cada técnica de forma cuantitativa.

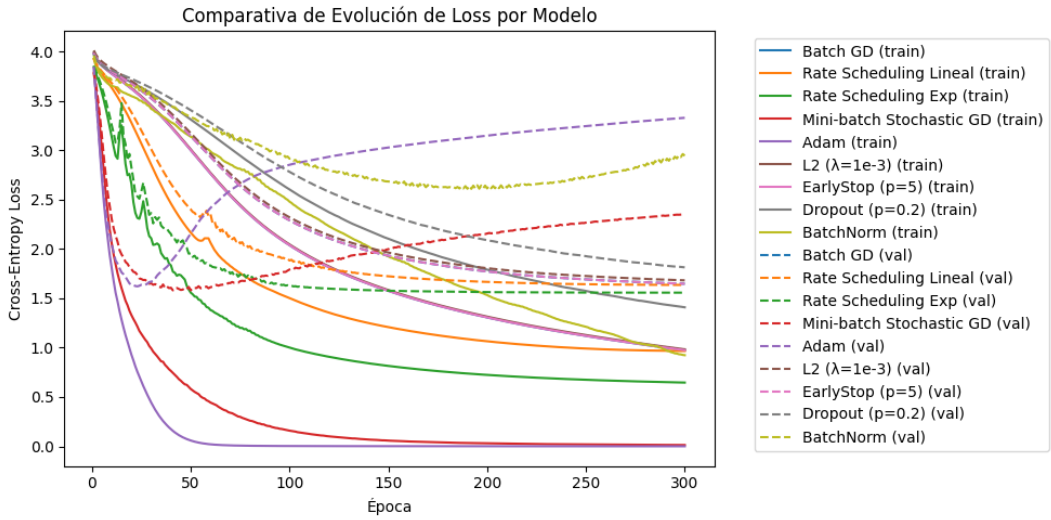


Figura 3: Evolución de la función de costo para cada técnica de optimización o regularización aplicada sobre $M0$.

Cuadro 1: Comparativa de Métricas

Método	**Acc Train**	**CE Train**	**Acc Val**	**CE Val**
Batch GD	0.7700	0.9751	0.5887	1.6482
Rate Scheduling Lineal	0.7678	0.9664	0.5975	1.6362
Rate Scheduling Exp	0.8588	0.6464	0.6275	1.5579
Mini-batch Stochastic GD	1.0000	0.0136	0.6188	2.3518
Adam	1.0000	0.0005	0.6288	3.3302
L2 ($\lambda = 10^{-1}$)	0.7700	0.9751	0.5887	1.6482
EarlyStop (p=5)	0.7700	0.9751	0.5887	1.6482
Dropout (p=0.2)	0.6869	1.4094	0.5550	1.8145
BatchNorm	0.7600	0.9241	0.3900	2.9454

De esta comparación aislada se observa que, entre las optimizaciones puras, Adam alcanza la mayor *accuracy* en validación ($\approx 62{,}9\,\%$), aunque a costa de una entropía cruzada muy baja en entrenamiento pero más alta en validación ($CE_{\text{val}} = 3{,}33$), lo que puede ser evidencia de un sobreajuste. La programación exponencial de la tasa de aprendizaje, por su parte, ofrece un buen balance: obtiene el costo más bajo ($CE_{\text{val}} = 1{,}5579$) y una buena *accuracy* en relación al resto ($Acc_{\text{val}} = 62{,}8\,\%$). La programación lineal mejora de forma moderada la estabilidad de la curva de pérdida y reduce ligeramente CE_{val} sin cambiar sustancialmente la precisión.

Por otro lado, las técnicas de regularización como dropout y batch normalization reducen la pérdida de entrenamiento pero, en este caso, de manera aislada y con los valores probados, no mejoran la *accuracy* de validación. Estos resultados sugieren que la combinación de estos mecanismos puede ayudar a mejorar la generalización en las siguientes arquitecturas.

3.2. Barrido de hiperparámetros: Modelo $M1$

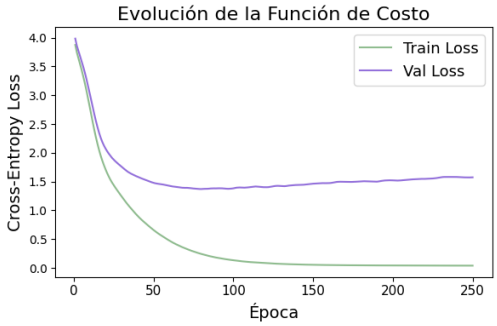
Siguiendo el procedimiento de búsqueda (*greedy search*) descrito en la Sección 2.8, se realizó un barrido sobre el espacio de arquitecturas y de hiperparámetros como detallado anteriormente, entrenando cada configuración durante 250 épocas. El objetivo fue optimizar la exactitud de validación y minimizar la función de costo sin realizar todas las combinaciones posibles de hiperparámetros, sino fijando paso a paso los mejores valores encontrados.

La mejor configuración hallada para el modelo $M1$ fue:

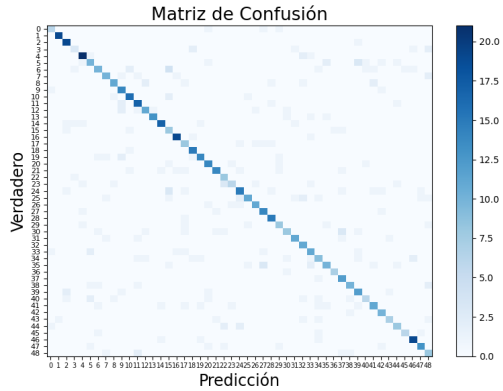
$$\text{Arquitectura: } [784 \text{ (input_size)},\ 512,\ 256,\ 49 \text{ (n_classes)}], \quad \text{Optimizador: Adam},$$
$$\eta = 0{,}001, \quad p_{\text{dropout}} = 0{,}30, \quad \text{BatchNorm: desactivado}, \quad \lambda_{L_2} = 0{,}1,$$
$$m = None, \quad \text{Early stopping: desactivado}, \quad \text{Lr Scheduler: ninguno}$$

Evaluado sobre el conjunto de validación, $M1$ alcanzó:

$$\text{Accuracy}_{\text{val}} = 0{,}7075, \quad \text{CE}_{\text{val}} = 1{,}4263.$$



(a) Evolución de la pérdida (*loss*) de entrenamiento y validación



(b) Matriz de confusión

Figura 4: Evaluación de $M1$ en el conjunto de validación

Este resultado supone una mejora de 9,87 puntos porcentuales en *accuracy* con respecto al modelo base $M0$ (Sección 3). Esto sugiere que la combinación de Adam, dropout y la regularización L2 aporta una mejora sustancial al desempeño del modelo. Además, la arquitectura resultante demuestra que aumentar el número de neuronas en las dos capas ocultas potencia la capacidad de representación de la red, y proporciona mayores beneficios que aumentar su profundidad, contribuyendo al incremento observado en la exactitud.

3.3. Entrenamiento en PyTorch: Modelo $M2$

Para validar la reproducibilidad y la eficiencia de la implementación manual, replicamos el modelo $M1$ en PyTorch usando la misma arquitectura y los hiperparámetros óptimos hallados (Sección 3.2).

El entrenamiento se realizó definiendo el mismo ciclo de entrenamiento y validación, midiendo la entropía cruzada y la exactitud (*accuracy*) en cada época. El modelo convergió sin señales de inestabilidad y alcanzó los siguientes resultados sobre el conjunto de validación:

$$\text{Accuracy}_{\text{val}} = 0{,}7150, \quad \text{CE}_{\text{val}} = 1{,}9740.$$

Estos valores muestran que la implementación en PyTorch no solo reproduce de forma bastante similar el modelo obtenido con el barrido de hiperparámetros (Sección 3.2), sino que además reduce significativamente el tiempo de entrenamiento por época (aproximadamente un 40 % menos).

3.4. Búsqueda de arquitectura: Modelo $M3$

Con los hiperparámetros óptimos ya fijados en el Modelo $M1$ (Sección 3.2), procedimos a explorar distintas configuraciones de profundidad y ancho de la red, sin alterar el resto de los parámetros, para determinar cuál estructura obtenía el mejor rendimiento a partir de los ajustes previos.

Las variantes probadas iban desde redes sencillas de dos capas ocultas de gran tamaño (por ejemplo, $[768, 384]$ y $[640, 320]$) hasta diseños de tres o cuatro capas intermedias con arquitecturas en "embudo" (por ejemplo, $[512, 256, 128]$, $[1024, 512, 256, 128]$) y configuraciones uniformes de alta capacidad ($[512, 512, 512]$, $[384, 384, 384]$), así como variantes con cuellos de botella intencionales ($[512, 128]$, $[512, 256, 64]$). En total se evaluaron veintidós combinaciones distintas, midiendo la *accuracy* en validación de cada una para un entrenamiento de 250 épocas.

De este barrido, la arquitectura que destacó fue

$$M3 : [784,\ 768,\ 384,\ 49],$$

que alcanzó una exactitud de validación de 0,7275 y una entropía cruzada de 1,9612. Este modelo mejora en casi 2 puntos porcentuales la *accuracy* obtenida por $M1$ en validación, demostrando que un aumento moderado en la capacidad de las dos capas ocultas favorece la representación de los caracteres sin caer en sobreajuste.

El Cuadro 2 resume las cinco mejores arquitecturas según *accuracy* en validación:

Cuadro 2: Top 5 arquitecturas según *accuracy* en validación para $M3$

Arquitectura	**Acc Val**	**CE Val**
[784, 768, 384, 49]	0.7275	1.9408
[784, 640, 320, 49]	0.7225	1.9009
[784, 512, 128, 49]	0.6950	1.9698
[784, 320, 320, 320, 49]	0.6925	2.3737
[784, 512, 512, 256, 49]	0.6887	2.4006

3.5. Modelo $M4$: exploración intensiva de *overfitting*

Para estudiar el efecto del sobreajuste en redes de gran capacidad sin técnicas de regularización, se entrenaron múltiples arquitecturas profundas utilizando PyTorch con una configuración base sin ningún tipo de mejora: descenso de gradiente puro (`GD`), sin *dropout*, sin *batch normalization*, sin regularización L_2, ni programación de tasa de aprendizaje o *early stopping*.

La Figura 6 muestra la evolución de la pérdida de entrenamiento y validación para cada arquitectura durante 200 épocas.

De todas las configuraciones evaluadas, la más extrema fue:

$$\text{Arquitectura } M4 : [784,\ 512,\ 512,\ 512,\ 512,\ 512,\ 512,\ 49],$$

con seis capas ocultas uniformes, que alcanzó durante las primeras 200 épocas:

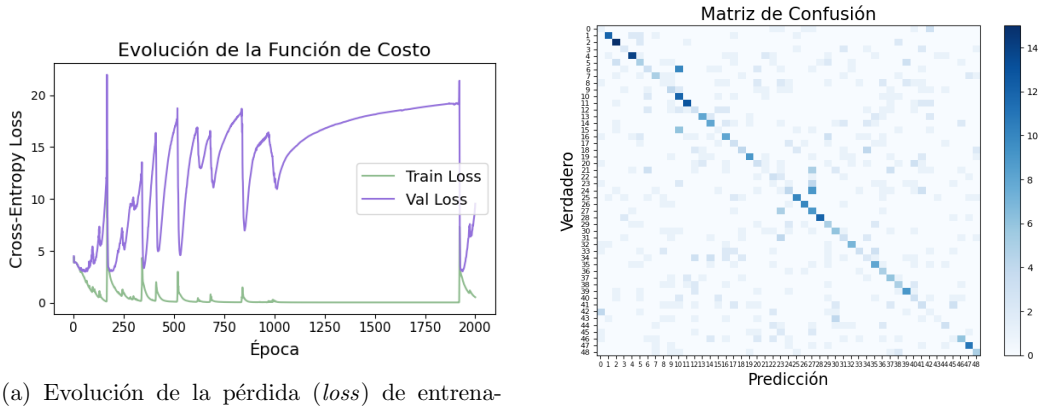
$$\text{Train loss} = 0{,}0509, \qquad \text{Val loss} = 19{,}4216.$$

Para profundizar el fenómeno, se extendió el mismo modelo a 2000 épocas con la arquitectura seleccionada. Los resultados finales fueron:

$$\text{Accuracy}_{\text{val},M4} = 0{,}3438, \qquad \text{CE}_{\text{val},M4} = 7{,}4190.$$

En la Figura 5 puede observarse el marcado contraste entre la evolución de la pérdida en entrenamiento (que se mantiene baja y relativamente más estable) y la de validación, la cual crece sostenidamente y presenta oscilaciones pronunciadas. Esta inestabilidad se debe a la alta varianza provocada por la gran capacidad de la arquitectura (seis capas profundas) combinada con la ausencia total de regularización: pequeñas fluctuaciones en el gradiente se amplifican y desestabilizan el ajuste, dando lugar a picos y valles en la curva de validación.

La matriz de confusión también evidencia una gran dispersión, reflejando errores sistemáticos en varias clases.



(a) Evolución de la pérdida (*loss*) de entrenamiento y validación

(b) Matriz de confusión

Figura 5: Evaluación de $M4$ en el conjunto de validación

Este comportamiento es característico del *overfitting* pronunciado: la red memoriza los ejemplos de entrenamiento pero pierde capacidad de generalización a nuevos datos. La falta de regularización y la gran arquitectura explican esta tendencia.

3.6. Evaluación de los modelos desarrollados

Finalmente, todos los modelos entrenados (M0–M4) se evaluaron en el conjunto de prueba. El Cuadro 3 resume la *accuracy* y la *cross-entropy* obtenidas:

De este análisis se concluye que el modelo $M3$ es el que mejor funciona: alcanza alcanza la mayor *accuracy* $(70,6\,\%)$ y manteniene una *cross-entropy loss* moderada $(2,08)$, lo que indica

Cuadro 3: Comparativa de resultados en el conjunto de Test

Modelo	**Accuracy**	**Cross-Entropy**
M0 (base propia)	0.6020	1.9409
M1 (mejor propia)	0.6770	1.5798
M2 (PyTorch réplica de M1)	0.6900	2.1878
M3 (mejor arquitectura PyTorch)	0.7060	2.0791
M4 (over-fitting)	0.3300	8.2271

que la combinación de su arquitectura [784, 768, 384, 49] con las técnicas de optimización y regularización utilizadas logra el mejor equilibrio entre capacidad y generalización.

En el otro extremo, el modelo $M4$, entrenado sin ninguna técnica de regularización, presenta el peor desempeño, con una *accuracy* de tan solo $33,0\,\%$ y una *cross-entropy* muy alta $(8,23)$, mostrando cómo una capacidad de red descontrolada perjudica gravemente la generalización. Estas métricas evidencian un sobreajuste extremo: la red memoriza el entrenamiento pero fracasa al generalizar.

Los modelos intermedios reflejan cómo cada mejora incremental aporta valor: El modelo $M1$, que utiliza Adam como optimizador junto con *dropout* y regularización *L2* con hiperparámetros ajustados, ya supera ampliamente al modelo base $M0$ (pasando de $60,2\,\%$ a $67,7\,\%$ de *accuracy*), lo que deja en evidencia el impacto positivo de estas técnicas. La réplica en PyTorch ($M2$) sube todavía más la exactitud $(69,0\,\%)$, aunque su pérdida de entropía es algo mayor que $M1$ ($2,1878$ vs. $1,5798$), lo cual sugiere que, si bien el modelo acierta más casos, sus probabilidades están menos calibradas.

En conjunto, estos resultados confirman que el balance entre capacidad de la red, acompañado de estrategias de regularización/optimización y programación de la tasa de aprendizaje, es esencial para obtener modelos robustos que generalicen bien a datos nuevos.

3.7. Predicciones finales

Una vez seleccionado el mejor modelo ($M3$, Sección 3.4), entrenado con los mejores hiperparámetros, se generaron las predicciones sobre el conjunto externo `X_COMP` (ver sección 5 del `notebook`). Para esto:

1. Se cargaron los pesos entrenados del modelo $M3$.
2. Se preprocesaron las imágenes de prueba de la misma forma que en entrenamiento (4).
3. Se ejecutó un *forward pass* para obtener, para cada muestra, el vector de probabilidades $\hat{\mathbf{y}} \in R^{49}$ (softmax en la capa de salida).
4. Se guardaron las probabilidades en el archivo `Tissera_AnaPaula_predicciones.csv`, con el formato:
$$\text{ID},\ p_1,\ p_2,\ \ldots,\ p_{49},$$
donde ID identifica la imagen y $p_k = \hat{y}_k$ es la probabilidad asignada a la clase k.

4. Apéndice

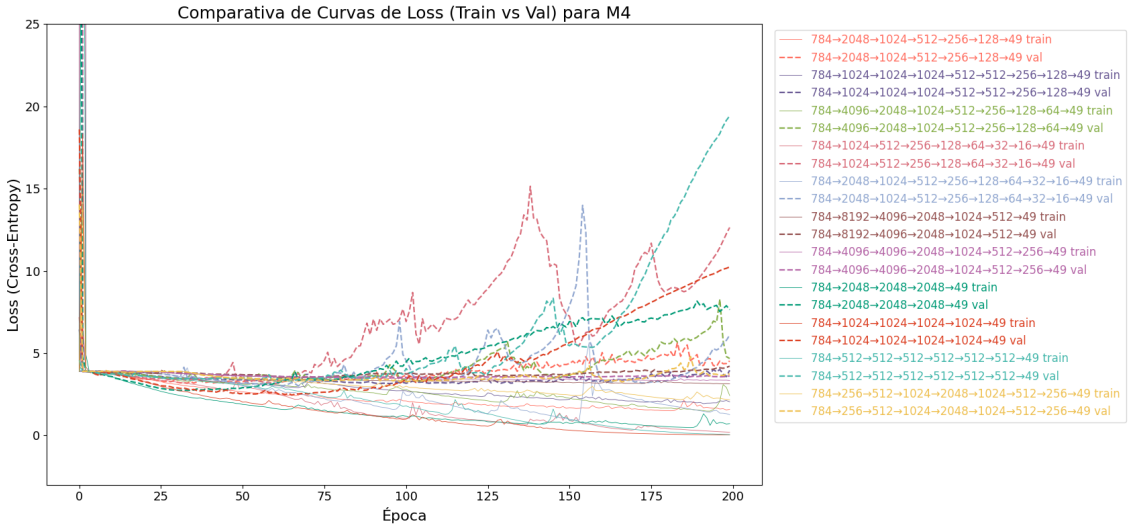


Figura 6: Comparativa de curvas de pérdida (train vs val) para las distintas arquitecturas bajo la configuración base de $M4$.